bioRχiv, 2025, 1–9
doi:
Original Article

PoolParty: streamlined design of DNA sequence libraries in Python

Zhihan Liu[1] and Justin B. Kinney[1,*]

[1]Simons Center for Quantitative Biology, Cold Spring Harbor Laboratory, 1 Bungtown Rd., 11724, New York, United States

*Corresponding author. jkinney@cshl.edu

FOR PUBLISHER ONLY Received on Date Month Year; revised on Date Month Year; accepted on Date Month Year

Abstract

Summary: Computationally designed DNA sequence libraries (also called oligonucleotide pools) are widely used in massively parallel reporter assays (MPRAs), protein deep mutational scanning (DMS) experiments, and other multiplexed assays of variant effect (MAVEs). Increasingly, such libraries are also being used to analyze and interpret genomic AI models in silico. Despite their importance, the process of designing these libraries remains tedious and error-prone due to the lack of purpose-built software. Here we introduce PoolParty, a Python package that enables the streamlined design of sequence libraries using a declarative, composable syntax. In PoolParty, users build libraries by chaining together a core set of operations and sequence transformations, yielding a directed acyclic graph (DAG). This makes it possible to specify complex library designs (e.g., mixing multiple types of systematic and random variation across multiple sequence regions) in just a few lines of code. PoolParty automatically generates informative names for each sequence, as well as "design cards" that describe the specific set of operations used to generate the sequence. Over 20 built-in operation types cover nucleotide- and codon-level mutagenesis, motif insertion, deletion and insertion scans, barcode generation, and more. Visualization methods let users quickly audit library content and inspect the DAG. PoolParty thus transforms oligonucleotide pool design from ad hoc scripting into a structured, transparent, and reproducible process.

Availability and implementation: PoolParty can be installed using the pip package manager and is compatible with Python $\geq$ 3.10. Documentation is provided at http://poolparty.readthedocs.io; source code is available at http://github.com/jbkinney/poolparty.

Contact: jkinney@cshl.edu (J.B.K.)

Introduction

Oligonucleotide pools have become essential tools in functional genomics. In massively parallel reporter assays (MPRAs), designed libraries of DNA sequences are routinely used to measure the regulatory activities of tens of thousands of candidate or variant regulatory elements in a single experiment [Inoue and Ahituv, 2015]. In deep mutational scanning (DMS) experiments, coding sequences that have been either systematically or randomly mutagenized are commonly used to map protein fitness landscapes [Fowler and Fields, 2014]. Both MPRAs and DMS experiments are examples of multiplexed assays of variant effect (MAVEs), a large and expanding family of high-throughput methods that quantitatively link sequence to function at scale [Kinney and McCandlish, 2019]. The same idea extends beyond the wet lab: complex DNA sequence libraries are increasingly being used in silico to probe the behavior of genomic AI models [Avsec et al., 2021, Nagai et al., 2026].

Computationally designing these libraries is straightforward in principle, but in practice it is tedious and error-prone. Typical applications require oligo pools that comprise multiple types of variant sequences generated in multiple ways. A DMS library, for example, might combine exhaustive single-amino-acid substitutions, sampled higher-order mutants, wild-type replicates, and barcodes — each generated by a different procedure and subject to different coverage requirements. An MPRA library might tile multiple transcription factor binding sites across a regulatory element in all possible orders and orientations, then assign each variant multiple barcodes. Coordinating this kind of combinatorial logic in a one-off script quickly becomes unwieldy, and subtle errors are hard to catch.

Existing tools can help with parts of the process, but none provide a unified framework for library design. General-purpose

© The Author 2025. Published by Oxford University Press. All rights reserved. For permissions, please e-mail: journals.permissions@oup.com

sequence toolkits [Cock et al., 2009, Schreiber, 2025, Pereira et al., 2015, Klie and Laub, 2023] can manipulate individual sequences but have no notion of a variant library as a structured object; in practice, users must write custom scripts to handle the combinatorial logic themselves. Assay-specific tools automate library design for particular experimental formats but are not easily adapted to novel designs [Georgakopoulos-Soares et al., 2016, Ghazi et al., 2018, Barbon et al., 2021]. And sequence optimization tools such as DnaChisel [Zulkower and Rosser, 2020] and CodonGenie [Swainston et al., 2017] address synthesis constraints on individual sequences, not library-level design.

Here we present PoolParty, a Python package that enables the streamlined design of complex sequence libraries. In PoolParty, libraries are represented as objects called Pools, while individual steps in the sequence generation process are called Operations. Using a concise and transparent syntax, users chain together Operations into a directed acyclic graph (DAG) that yields the desired Pool. The DAG describes the high-level logic used to generate sequences, while the procedural logic and bookkeeping are handled internally. Importantly, no sequences are generated until the user requests them, and only the requested sequences are generated. This makes it possible for users to explore and test multiple design options without triggering expensive computation. When sequences are generated, each is automatically assigned an informative name and paired with a *design card* that records the specific procedures used to construct it. We demonstrate PoolParty in three example applications: a DMS library for protein GB1 modeled after Olson et al. [2014], an MPRA library probing transcription factor binding site grammar, and an in silico experiment characterizing the behavior of SpliceAI [Jaganathan et al., 2019] in which the design cards form the basis for surrogate modeling.

Methods

Pools and Operations

PoolParty is built around two core concepts: Pools and Operations. Pools represent collections of sequences—either the final library that the user wishes to generate, or intermediate sets of sequences out of which the final library is constructed. Operations represent the steps used to create Pools. When designing a library, users chain together multiple Operations into a DAG that yields the desired Pool. Sequences are then generated on demand from the final Pool using the DAG. Instead of generating sequences directly, users can test many different library designs — inspecting a few sampled sequences from each — before committing to generating the full library.

PoolParty provides over 20 built-in Operations that users can choose from when constructing DAGs. Each Operation takes zero or more Pools as input and returns exactly one Pool as output. It is useful to think of Operations as falling into four functional categories (Fig. 1A): Source Operations, Transformation Operations, Composition Operations, and State Operations. *Source Operations* create the initial Pools from which all downstream Pools are constructed, e.g., user-specified sequences, DNA sequences generated using position weight matrices or IUPAC motifs, random k-mers, or barcodes. *Transformation Operations* modify sequence content, e.g., generating sequence variants through point mutations, insertions, deletions, recombination, or shuffling. Specialized codon-aware Operations are provided for transformations that operate on open reading frames. *Composition Operations* combine sequences from multiple parent Pools, either by concatenating parent sequences end-to-end (join) or by merging multiple Pools into one Pool (stack). *State Operations* affect which sequences are in a Pool and how they are ordered, and include methods to select, reorder, and replicate sequences.

State Tracking and Sequence Generation

To generate a sequence, PoolParty passes a *state* — an integer index that, together with a random seed, uniquely determines the output — to the *root Pool* (i.e., the final Pool from which sequences are requested). Generation then proceeds in two passes (Fig. 1B). In the backward pass, PoolParty propagates the state from the root Pool back through the DAG. Each Operation decomposes the state it receives into an internal state and one or more states that are then passed to its input Pools. Each of the input Pools then passes its assigned state to its upstream Operation. In the forward pass, each Operation generates a sequence based on its input sequences (if any) and its internal state. This process starts at the source Operations and proceeds forward until the root Pool receives the completed sequence.

Users control how each Operation resolves its design choices through one of two generation modes. Sequential mode enumerates all possibilities exhaustively, such as all single-point mutations at a set of positions. Random mode samples from the specified design space. The resulting choices are recorded as internal states of the Operation. These internal states then compose with upstream Pool states to produce the states of downstream Pools. When a DAG mixes both modes, tracking states to ensure complete and reproducible library coverage becomes difficult as the design scales.

To address this, we developed StateTracker, a backend package that defines composition rules governing how states combine across the DAG. The default rule is the Cartesian product: each downstream Pool state corresponds to a unique combination of upstream Pool and Operation states, and can thus be decomposed into its constituents through mixed-radix decomposition. For stack Operations, derived Pool states are the disjoint union of all parent Pool states, so each downstream state corresponds to exactly one state of a parent. Additional rules exist to handle Operations such as sampling, slicing, shuffling, and repetition.

During library specification, these composition rules define the state structure of each Pool, and the state count of the final Pool defines the default library size. Sequential-mode Operations contribute their full enumeration, whereas random-mode Operations contribute only when an explicit state count is specified, fixing a reproducible set of sampled choices as states. No sequences are generated at this stage.

To generate sequences, PoolParty iterates through the states of the final Pool. Setting a state value triggers StateTracker to propagate it backward through the DAG, successively resolving parent and Operation states at each node (Fig. 1B). Sequence construction then proceeds forward through the DAG, starting from source Operations. Each Operation reads its resolved state, produces its output, and passes it to the next Operation until the final library sequence is assembled (Fig. 1B). Random-mode Operations without a specified state count sample their design choices stochastically during this forward pass. A master seed ensures reproducibility.

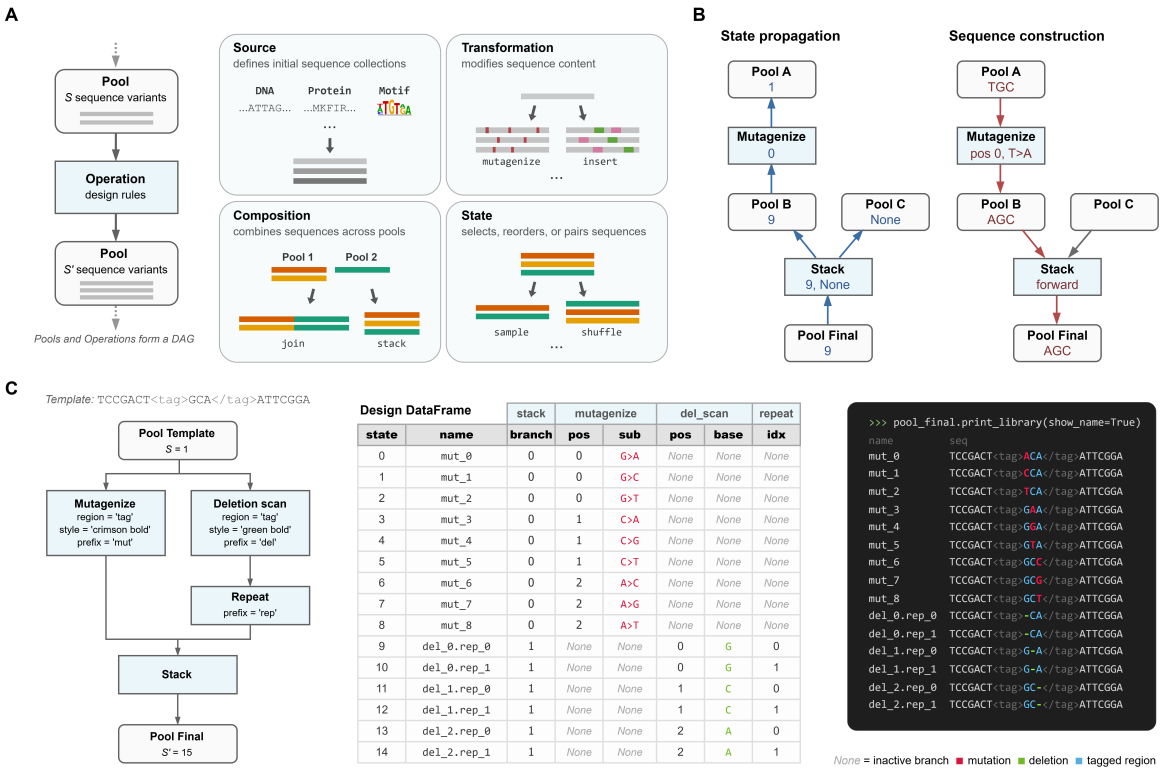

Fig. 1: **PoolParty framework for sequence library design. (A)** Library design workflows are represented as directed acyclic graphs (DAGs) of Pools and Operations (left). Pools represent collections of sequences, and Operations generate new Pools. Operations fall into four categories based on how they introduce, modify, combine, or reindex sequences: source, transformation, composition, and state (right). **(B)** Sequence generation for a given state of the final Pool occurs in two phases. The desired final sequence state is propagated backward through the DAG, thereby resolving the corresponding states of the upstream Pools and Operations (left). The sequence is then constructed via a forward pass from source Pools using the resolved states of each Operation (right). Nodes that receive no state (None) are inactive and do not contribute to sequence construction (grey arrows). **(C)** Example workflow. A template sequence containing a tagged region (`<tag>GCA</tag>`, $S = 1$) undergoes mutagenesis and deletion scanning in parallel. Deletion variants are replicated by `repeat`, and the outputs of both operations are combined by `stack` ($S = 15$). Left: workflow DAG. Center: design DataFrame returned by `generate_library`; columns are grouped by operation. Right: styled `print_library` output showing the tagged region in steel blue (via `.stylize()`).

Region Markers

Tracking flanking sequences and region boundaries across steps is error-prone when a design requires manipulations on different parts of a sequence. PoolParty avoids this by letting users specify a positional interval as the target region, such that the Operation is only applied to the interval while preserving the flanks. Alternatively, target regions can be defined by inserting XML-style tags into the sequence (Fig. 1C). Two types of tags are supported: content tags enclose an existing sub-sequence (e.g., `AAA<cre>CCCGGG</cre>TTT`), and self-closing tags mark insertion points (e.g., `ACGT<ins/>ACGT`). Operations can then target these regions by tag name (here, “cre” and “ins”). Named tags persist through the DAG, so multiple Operations can target the same or different named regions of a sequence repeatedly. Tags remain valid when upstream Operations change the content or length within a region.

Sequence Provenance

PoolParty has three complementary features to record sequence provenance.

Design cards record the design choices used to generate each sequence in the library. Users specify which Operations to track, and the resulting cards are returned as a DataFrame at the same time the sequences are generated. Each row corresponds to an individual sequence, and its columns record the design choices of the tracked Operations. Additional user-defined sequence properties can also be computed and returned as part of this DataFrame. As design cards are metadata encoding sequence features, they support downstream analyses such as filtering, grouping, and modeling without the need for custom sequence parsing.

Composite sequence names collectively summarize the states of the Operations used to generate each sequence. Users assign prefixes to each Operation they wish to track. During sequence generation, PoolParty combines each prefix with the corresponding resolved state and concatenates these prefix–state

pairs in topological order within the DAG to form the name. For example, `wt.mut_03.bc_01` denotes a wild-type sequence modified with mutation variant 3 and tagged with barcode 1. Operations without prefixes or on inactive branches are omitted.

Sequence styling enables PoolParty to render sequences with visual annotations that indicate their construction history. Operations can optionally annotate sequence positions they act on with different colors or text formatting (e.g., case changes). Annotations propagate through the same forward pass used for sequence construction, so visual styling from successive Operations layer on top of each other. For example, a mutated base can be highlighted in red within an inserted motif shown in blue. In ORF-aware designs, neighboring codons can be displayed with alternating colors to indicate the reading frame. Using the PoolParty helper method for translation, the resulting amino acids inherit the styling of their source codons.

Implementation and extensibility

PoolParty is implemented in pure Python and requires Python 3.10 or later. The package has minimal dependencies (NumPy for numerical operations) and is distributed under an open-source license via PyPI. Comprehensive documentation, tutorials, and example notebooks are available at the project website.

The architecture is designed for extensibility. New Operation types can be created by subclassing the base Operation class and implementing a small number of methods (state space size, sequence generation, metadata extraction). Custom Operation automatically inherit support for algebraic composition, global-state indexing, design card generation, and visualization, allowing researchers to extend PoolParty with domain-specific operations without reimplementing core infrastructure.

SpliceAI Surrogate Analysis

Cryptic 5′ splice-site (5′ss) 9-mers were sampled from the human 5′ss position weight matrix [Wong et al., 2018] and scored with MaxEntScan [Yeo and Burge, 2004]. Sequences were binned into 100 MaxEntScan score (hereafter, strength) quantiles, and 20 sequences were drawn from each bin (2,000 total). Each 9-mer was inserted at 100 positions flanking the SMN2 exon 7 canonical donor and paired with a matched control in which the GT dinucleotide was disrupted (T>A at +2; Fig. 4A).

SpliceAI donor probabilities at the canonical donor were predicted for all sequences using the five-model ensemble of Jaganathan et al. [2019] with ±5 kb of genomic context (GRCh38). For each library-control pair, we computed the difference in logit-transformed predictions at the canonical donor. These per-sequence values were averaged within each cell of the resulting 100×100 position $\times$ strength grid (Fig. 4B).

Generalized additive models (GAMs) were fit to the cell means on the grid using pyGAM [Servén and Brummitt, 2018], with smooth terms for position, strength, and their interaction. Models were weighted by the inverse variance of each cell mean. Exonic and intronic regions were modeled separately. Model performance was evaluated by checkerboard cross-validation of the grid.

Example Applications

Codon-aware DMS library design

A common use of designed oligonucleotide pools is for DMS. In these experiments, a specific open reading frame of interest is mutagenized to introduce single amino acid substitutions at each position, and sometimes at multiple positions, possibly with additional codon recoding. In one experiment, Olson et al. [2014] designed a library in which they mutagenized 55 residues of protein GB1, the IgG-binding domain of protein G. This yielded a library of ∼500k sequences, with near-complete coverage of single and double mutants. The pairwise mutations have proven to be particularly valuable and serve as the basis for studying the epistatic landscape [Zhou et al., 2022]. The nature of this dataset, together with the biophysics underlying GB1 function, has made it a classic testbed for quantitative sequence-to-function modeling [Tareen et al., 2022, Faure and Lehner, 2024].

We reconstruct and extend this library in PoolParty by defining a template Pool (`orf_pool`) from the GB1 coding sequence and applying four parallel Operations, each targeting a different mutational order (Fig. 2A,B). For each `mutagenize_orf` Operation, we specify only the substitution type (missense) and target positions (55 internal codons); PoolParty resolves the valid substitutions at each wild-type codon to their nucleotide changes. The `single_pool` contains all 1,045 single substitutions, and the `double_pool` contains all 546,085 pairwise combinations. A third `mutagenize_orf` Operation produces the `random_pool` of 10,000 variants at a 10% per-codon substitution rate, enriched for higher-order mutants. The `repeat` Operation produces a `wt_pool` of 100 wild-type replicates. The four Pools are combined via `stack` into the `dms_pool` (Fig. 2A).

The three `mutagenize_orf` Operations use two different generation modes, specified as a parameter on each Operation (Fig. 2B). The `single_pool` and `double_pool` Operations run in sequential mode, which exhaustively enumerates the specified design space. The `random_pool` Operation runs in random mode, sampling from the design space and fixing the result to a reproducible set. StateTracker resolves the state structure from the generation modes and design specifications of each Operation, without generating any sequences, so each of the 557,230 states in the `dms_pool` indexes into exactly one variant from one of the four source Pools. The full library specification can also be inspected as a DAG via `print_dag()` (Fig. 2D).

We set `report_design_cards=True` in `generate_library` so that each variant's generation history is tracked alongside its sequence (Fig. 2B). By default, every Operation is tracked, and each one adds its own set of design card features as columns in the output DataFrame. In Fig. 2C, each row is a library variant, and its columns record the design card features used in its generation: `stack` identifies the source Pool, `mutagenize_orf` records the codon positions, codon changes, and amino acid substitutions, and `repeat` records the replicate index.

MPRA library design for regulatory grammar

MPRAs use synthetic DNA sequences to dissect the cis-regulatory codes underlying a variety of gene regulatory steps. Unlike DMS experiments, MPRAs for regulatory grammar often involve systematically changing the spacing, order, orientation, and multiplicity of known regulatory motifs within an otherwise inert sequence. The combinatorial variation of these design parameters reveals how regulatory proteins bound at cognate sites orchestrate regulatory activity. For example, Georgakopoulos-Soares et al. [2023] tested over 200,000 sequences with all pairwise and triplet arrangements of 18 TFBSs in synthetic enhancers, and found that orientation and order are major drivers of expression. The scale

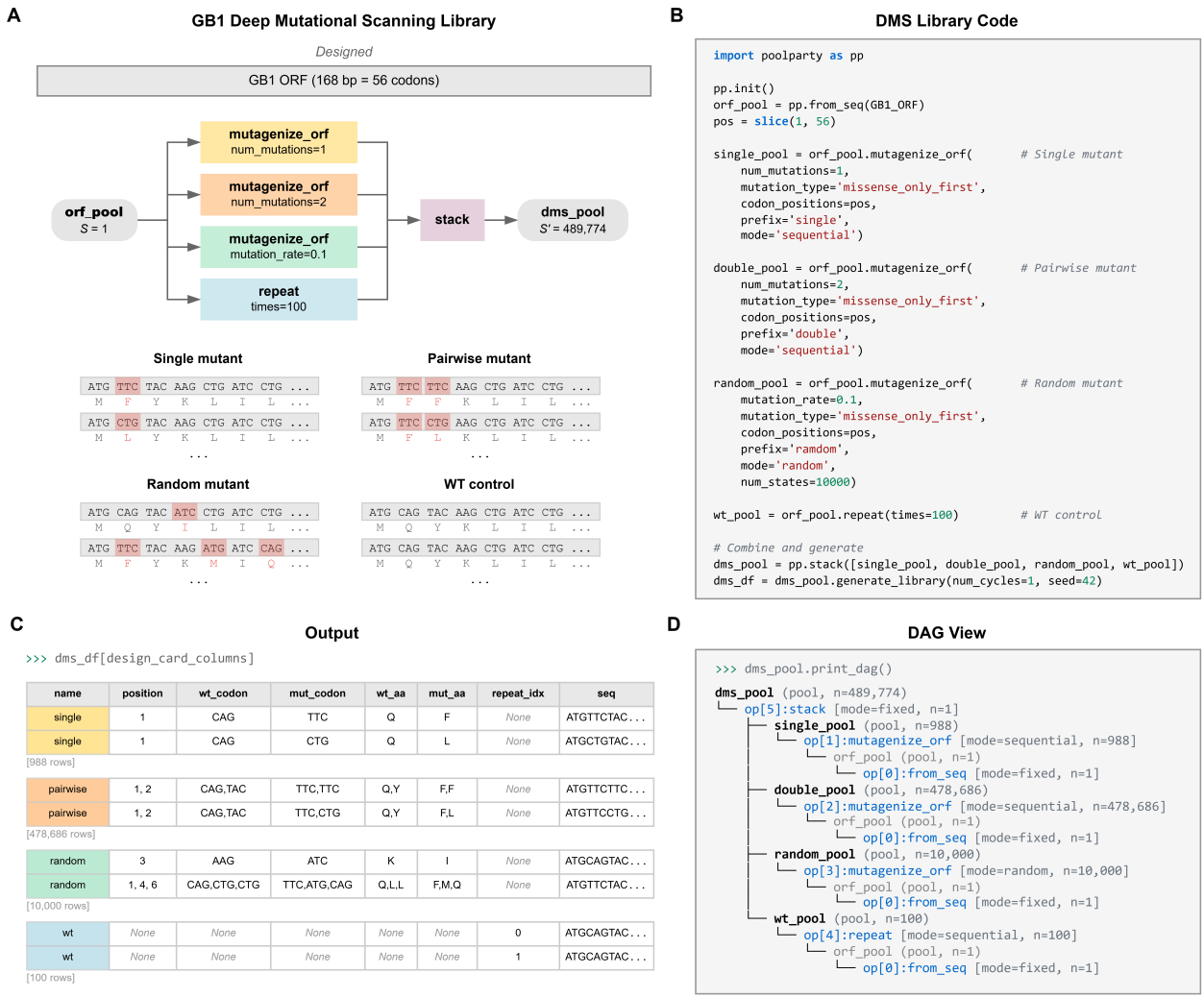

Fig. 2: **Design of a GB1 deep mutational scanning library using PoolParty. (A)** Schematic of the codon-aware mutational library for the GB1 ORF (168 bp; 56 codons). Parallel operations generate single- and pairwise-substitution pools, a random mutagenesis pool, and wild-type replicates, which are combined by stack to yield the final library (489,774 sequences). Representative sequence segments for each component are shown, with mutated codons highlighted and amino acid translations shown beneath. **(B)** Python code implementing the library design. Operations applied to the template sequence generate component pools, which are combined and materialized using `generate_library`. **(C)** Example design cards. Representative rows are shown for each component with the fields relevant to that component, and row counts per component are indicated. **(D)** DAG view of the library specification (`print_dag()`), showing the hierarchy of Pools and Operations.

and controlled design of these data also make them highly suited for training and benchmarking machine learning models [Fleur et al., 2024].

As a second use case, we designed an MPRA library studying how binding site arrangements of three liver-enriched TFs affect regulatory activity (Fig. 3A,B). Following the oligo layout of Melnikov et al. [2012], we define the template Pool from a sequence containing an inert 100 bp CRE region and a barcode region, both tagged with region markers (Section 2.3). Each of the three TFs — HNF4A, PPARA, and XBP1 — is represented by a Pool containing both the forward and reverse-complement binding sites. We use `replacement_multiscan` in sequential mode to tile these Pools within the central 80 bp of the CRE without overlap. PoolParty automatically enumerates 2,184 valid placements, each in eight orientation combinations, for a CRE Pool of 17,472 variants. After repeating this Pool three times, we fill the barcode region with random barcodes from `get_barcodes`, giving the final `mpra_pool` of 87,360 sequences at a 1:3 CRE-to-barcode ratio (Fig. 3A).

PoolParty supports two types of styling. The style parameter of Operations marks the modifications they introduce to the sequence, while the `stylize` Operation annotates a region of the existing sequence. We first use `stylize` to add a background color to the tagged CRE region in the template. For the inserts, we color each TFBS Pool using the style parameter in `from_seqs` (HNF4A blue, PPARA purple, XBP1 orange) and mark barcodes bold in

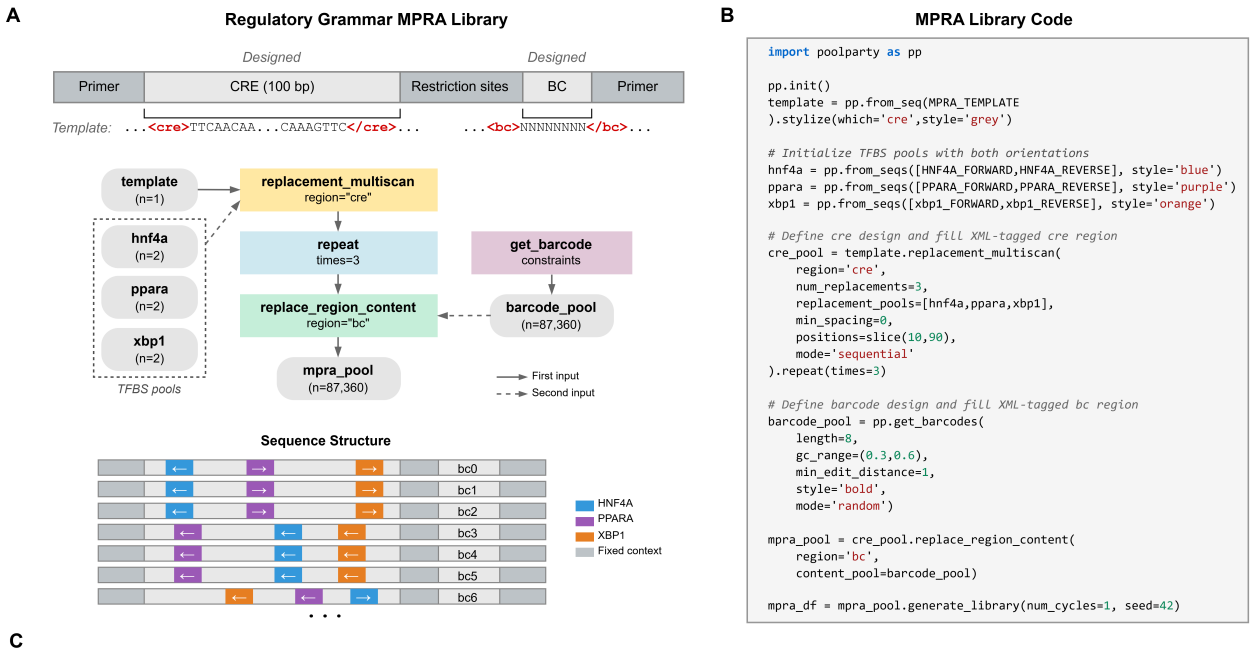

Fig. 3: **Design of a regulatory grammar MPRA library using PoolParty. (A)** Schematic of the regulatory grammar MPRA library targeting a 100-bp CRE. HNF4A, PPARA, and XBP1 binding sites are tiled across the CRE in different orders and orientations, with each variant assigned three unique barcodes (87,360 sequences total). Representative sequence structures are shown, with TFBS placements and barcode assignments indicated. **(B)** Python code for library construction. TFBS pools are initialized with both forward and reverse-complement sequences to represent orientation. **(C)** Example design cards. Representative rows are shown for each CRE variant, with insertion position, orientation, and barcode index recorded.

`get_barcodes` (Fig. 3B). When the TFBSs are tiled into the CRE, their colors overlay the CRE region background color, and both are rendered in the `print_library` output (Fig. 3C), making the positions and ordering of the three TFBSs immediately visible.

Surrogate modeling of SpliceAI predictions

In silico libraries designed in the same spirit as MPRAs are increasingly used to probe the regulatory mechanisms learned by genomic deep learning models, treating these models as virtual assays [Avsec et al., 2021, Toneyan and Koo, 2024, Seitz et al., 2024, Nagai et al., 2026]. After scoring these libraries with the model, the relationships between perturbed sequence features and predicted outcomes can be summarized by functions of simpler mathematical form that are intrinsically interpretable, an approach called surrogate modeling. PoolParty streamlines this process through design cards, which record the sequence changes defining each variant and can serve directly as covariates for surrogate models. As a third use case, we apply this approach to characterize the effect of cryptic donor insertion on canonical 5'ss strength as judged by SpliceAI [Jaganathan et al., 2019].

Using PoolParty, we designed a library by varying the strength (MaxEntScan) of inserted cryptic donors and their position relative to the canonical 5'ss of SMN2 exon 7 (Methods). The library covered a full grid of 100 positions and 100 strength quantile bins, with design cards automatically recording these parameters for each variant (Fig. 4A). We paired each variant with a matched control and defined the cryptic donor effect as the difference in SpliceAI predictions at the canonical 5'ss between the two. Separate generalized additive models (GAMs) were fit to this quantity for exonic and intronic regions, using only smooth spline terms for strength, position, and their interaction (Fig. 4A).

The GAMs achieved a held-out R^2 of 0.82 (exon) and 0.71 (intron), and their response surfaces matched the observed data (Fig. 4B,C). Positional effects showed stronger suppression for exonic than intronic cryptic sites (Fig. 4D), with effects peaking at 20 bp upstream, consistent with a prior report of preferential depletion of decoy donors proximal to annotated 5'sss on the exonic side [Dawes et al., 2022]. In terms of strength effects, stronger cryptic donors drove larger suppression in both models (Fig. 4E). Positional and strength effects also interacted in a non-additive manner, as shown by the interaction surface (Fig. 4F). We compared the GAMs to simpler surrogate alternatives and found that incorporating non-linearity and interaction terms enhanced predictive performance (Fig. 4G).

Discussion

PoolParty is a Python package for the declarative design of oligonucleotide libraries, providing an alternative to the ad hoc scripting that often underlies library construction workflows. It abstracts sequence collections and individual design steps as composable programming objects (Pools and Operations, respectively) that can be assembled into a DAG from a high-level specification, supporting complex libraries

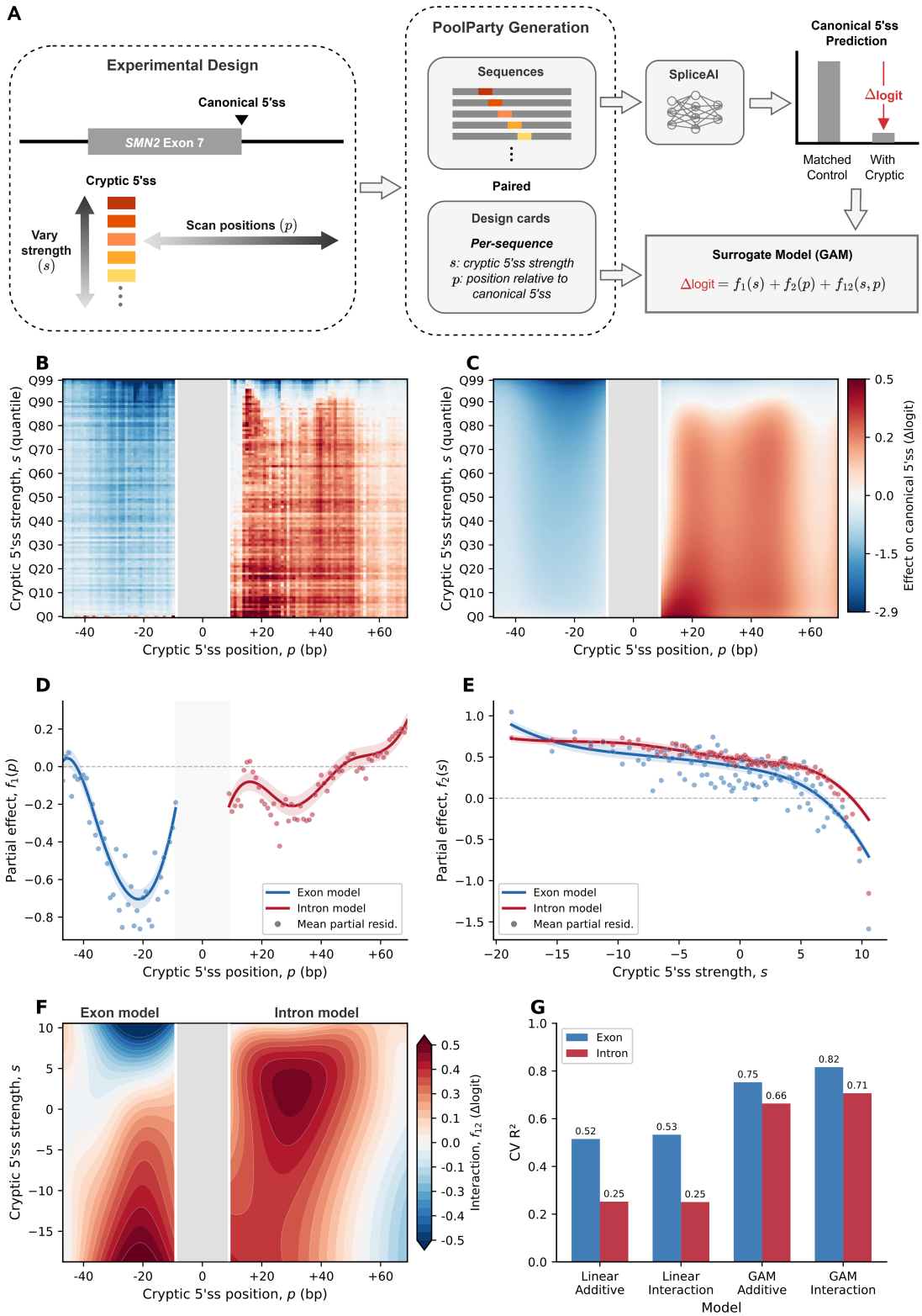

Fig. 4: **Surrogate modeling of SpliceAI predictions using design-card features. (A)** Experimental design. Cryptic 5′ss 9-mers spanning a range of MaxEntScan strengths (s) are inserted at positions (p) flanking the SMN2 exon 7 canonical donor. Paired library (GT) and control (GA) sequences are generated by PoolParty, and s and p are recorded in design cards. SpliceAI predictions at the canonical donor are summarized as Δlogit and modeled as a function of s and p using a generalized additive model (GAM). **(B)** Mean Δlogit across the strength × position grid (100 × 100; each cell averages 20 distinct 9-mers). The grey band marks the canonical donor region. **(C)** GAM interaction model predictions using the same color scale as in B. **(D)** Partial dependence of the position term $f_1(p)$, shown separately for exon (blue) and intron (red) models. Shaded regions indicate 95% confidence intervals; points denote mean partial residuals. **(E)** Partial dependence of the strength term $f_2(s)$, with conventions as in D. **(F)** Interaction surface $f_{12}(s,p)$ for exon and intron models. **(G)** Block cross-validation R^2 for linear and GAM models, with and without interaction terms.

without error-prone manual bookkeeping. A distinctive feature of PoolParty is that each variant in the library can optionally carry a structured record of the design choices that produced it as machine-readable metadata. We demonstrated these capabilities in three functional genomics applications, namely codon-aware mutagenesis, regulatory element tiling, and in silico perturbation experiments, covering substantially different experimental objectives and constraints.

Several features merit mention. The per-Operation separation of sequential and stochastic generation allows exhaustive and sampled components to coexist in a single library specification, avoiding the need to build and merge separate libraries. ORF-aware Operations preserve reading frames and handle conversion between DNA and protein sequences, which eliminates common sources of errors in DMS library design. Sequence styling and DAG visualization provide immediate verification on library architecture before synthesis. Design cards enable model predictions to be analyzed directly in terms of explicit sequence features; in the surrogate modeling example, these features served as covariates for an interpretable model, and we expect this to generalize to systematic and scalable interrogation of genomic deep learning models.

PoolParty does have limitations. The current implementation is optimized for libraries of moderate size (up to millions of sequences) generated on a single machine; very large design spaces may require distributed computation or streaming generation, which are not yet supported. Additionally, PoolParty focuses on sequence generation rather than synthesis optimization; users who require GC content balancing, homopolymer avoidance, or other synthesis constraints should apply post-processing filters or use complementary tools such as DnaChisel [Zulkower and Rosser, 2020].

Looking forward, we envision several extensions. Integration with constraint solvers could enable joint optimization of library coverage and synthesis feasibility. Support for paired-end and long-read sequencing adapters would streamline library preparation workflows. Finally, a web-based interface could make PoolParty accessible to users without Python programming experience.

In summary, PoolParty transforms oligonucleotide pool design from an error-prone, ad hoc process into a structured, transparent, and reproducible workflow. We anticipate that it will accelerate the design and execution of MPRAs, DMS experiments, and other high-throughput functional genomics assays.

Competing interests

The authors declare no competing interests.

Author contributions statement

Z.L concieved the project. J.B.K designed the project, wrote the software, and wrote the manuscript. J.B.K supervised the research.

Acknowledgments

This work was supported in part by: NIH grants R01HG011787 (J.B.K, Z.L.). Computational equiptment was supported by NIH grant S10OD028632.

References

Žiga Avsec, Melanie Weilert, Avanti Shrikumar, Sabrina Krueger, Amr Alexandari, Khyati Dalal, Robin Fropf, Charles McAnany, Julien Gagneur, Anshul Kundaje, and Julia Zeitlinger. Base-resolution models of transcription-factor binding reveal soft motif syntax. *Nature Genetics*, 53(3):354–366, March 2021. ISSN 1546-1718. doi: 10.1038/s41588-021-00782-6.

Luca Barbon, Victoria Offord, Elizabeth J Radford, Adam P Butler, Sebastian S Gerety, David J Adams, Hong Kee Tan, and Andrew J Waters. Variant library annotation tool (VaLiAnT): an oligonucleotide library design and annotation tool for saturation genome editing and other deep mutational scanning experiments. *Bioinformatics*, 38(4):892–899, 2021. ISSN 1367-4803. doi: 10.1093/bioinformatics/btab776.

Peter J. A. Cock, Tiago Antao, Jeffrey T. Chang, Brad A. Chapman, Cymon J. Cox, Andrew Dalke, Iddo Friedberg, Thomas Hamelryck, Frank Kauff, Bartek Wilczynski, and Michiel J. L. de Hoon. Biopython: freely available python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11):1422–1423, 2009. ISSN 1367-4803. doi: 10.1093/bioinformatics/btp163.

Ruebena Dawes, Himanshu Joshi, and Sandra T. Cooper. Empirical prediction of variant-activated cryptic splice donors using population-based RNA-Seq data. *Nature Communications*, 13(1):1655, March 2022. ISSN 2041-1723. doi: 10.1038/s41467-022-29271-y.

Andre J. Faure and Ben Lehner. MoCHI: Neural networks to fit interpretable models and quantify energies, energetic couplings, epistasis, and allostery from deep mutational scanning data. *Genome Biology*, 25(1):303, December 2024. ISSN 1474-760X. doi: 10.1186/s13059-024-03444-y.

Alyssa La Fleur, Yongsheng Shi, and Georg Seelig. Decoding biology with massively parallel reporter assays and machine learning. *Genes & Development*, 38(17-20):843–865, September 2024. ISSN 0890-9369, 1549-5477. doi: 10.1101/gad.351800.124.

Douglas M Fowler and Stanley Fields. Deep mutational scanning: a new style of protein science. *Nat Methods*, 11(8):801 – 807, 2014. ISSN 1548-7105. doi: 10.1038/nmeth.3027.

Ilias Georgakopoulos-Soares, Naman Jain, Jesse M Gray, and Martin Hemberg. MPRAnator: a web-based tool for the design of massively parallel reporter assay experiments. *Bioinformatics*, 33(1):137–138, 2016. ISSN 1367-4803. doi: 10.1093/bioinformatics/btw584.

Ilias Georgakopoulos-Soares, Chengyu Deng, Vikram Agarwal, Candace S. Y. Chan, Jingjing Zhao, Fumitaka Inoue, and Nadav Ahituv. Transcription factor binding site orientation and order are major drivers of gene regulatory activity. *Nature Communications*, 14(1):2333, April 2023. ISSN 2041-1723. doi: 10.1038/s41467-023-37960-5.

Andrew R Ghazi, Edward S Chen, David M Henke, Namrata Madan, Leonard C Edelstein, and Chad A Shaw. Design tools for MPRA experiments. *Bioinformatics*, 34(15):2682–2683, 2018. ISSN 1367-4803. doi: 10.1093/bioinformatics/bty150.

Fumitaka Inoue and Nadav Ahituv. Decoding enhancers using massively parallel reporter assays. *Genomics*, 106(3):159 – 164, 2015. ISSN 0888-7543. doi: 10.1016/j.ygeno.2015.06.005.

Kishore Jaganathan, Sofia Kyriazopoulou Panagiotopoulou, Jeremy F. McRae, Siavash Fazel Darbandi, David Knowles, Yang I. Li, Jack A. Kosmicki, Juan Arbelaez, Wenwu Cui, Grace B. Schwartz, Eric D. Chow, Efstathios Kanterakis, Hong

Gao, Amirali Kia, Serafim Batzoglou, Stephan J. Sanders, and Kyle Kai-How Farh. Predicting Splicing from Primary Sequence with Deep Learning. *Cell*, 176(3):535–548.e24, January 2019. ISSN 0092-8674, 1097-4172. doi: 10.1016/j.cell.2018.12.015.

Justin B Kinney and David M McCandlish. Massively parallel assays and quantitative sequence-function relationships. *Annual Review of Genomics and Human Genetics*, 20(1): 99–127, 08 2019. ISSN 1527-8204. doi: 10.1146/annurev-genom-083118-014845. Wrote.

Adam Klie and David Laub. SeqPro (sequence processing toolkit), 2023. URL `https://github.com/ML4GLand/SeqPro`.

Alexandre Melnikov, Anand Murugan, Xiaolan Zhang, Tiberiu Tesileanu, Li Wang, Peter Rogov, Soheil Feizi, Andreas Gnirke, Curtis G Callan, Justin B Kinney, Manolis Kellis, Eric S Lander, and Tarjei S Mikkelsen. Systematic dissection and optimization of inducible enhancers in human cells using a massively parallel reporter assay. *Nature Biotechnology*, 30(3):271–277, 02 2012. ISSN 1087-0156. doi: 10.1038/nbt.2137. Wrote (coauthored).

Masayuki Nagai, Alan E. Murphy, Kaeli Rizzo, and Peter K. Koo. Toward Interpretable and Generalizable AI in Regulatory Genomics, February 2026.

C Anders Olson, Nicholas C Wu, and Ren Sun. A comprehensive biophysical description of pairwise epistasis throughout an entire protein domain. *Current biology : CB*, 24(22):2643 – 2651, 11 2014. ISSN 0960-9822. doi: 10.1016/j.cub.2014.09.072.

Filipa Pereira, Flávio Azevedo, Ângela Carvalho, Gabriela F Ribeiro, Mark W Budde, and Björn Johansson. Pydna: a simulation and documentation tool for DNA assembly strategies using python. *BMC Bioinformatics*, 16(1):142, 2015. doi: 10.1186/s12859-015-0544-x.

Jacob Schreiber. tangermeme: A toolkit for understanding cis-regulatory logic using deep learning models. *bioRxiv*, page 2025.08.08.669296, 2025. doi: 10.1101/2025.08.08.669296.

Evan E. Seitz, David M. McCandlish, Justin B. Kinney, and Peter K. Koo. Interpreting cis-regulatory mechanisms from genomic deep neural networks using surrogate models. *Nature Machine Intelligence*, 6(6):701–713, June 2024. ISSN 2522-5839. doi: 10.1038/s42256-024-00851-5.

Daniel Servén and Charlie Brummitt. pyGAM: Generalized additive models in python, March 2018. URL `https://doi.org/10.5281/zenodo.1208723`.

Neil Swainston, Andrew Currin, Lucy Green, Rainer Breitling, Philip J Day, and Douglas B Kell. CodonGenie: optimised ambiguous codon design tools. *PeerJ Computer Science*, 3: e120, 2017. doi: 10.7717/peerj-cs.120.

Ammar Tareen, Mahdi Kooshkbaghi, Anna Posfai, William T. Ireland, David M. McCandlish, and Justin B. Kinney. MAVE-NN: learning genotype-phenotype maps from multiplex assays of variant effect. *Genome Biology*, 23(1):98, 2022. ISSN 1474-7596. doi: 10.1186/s13059-022-02661-7.

Shushan Toneyan and Peter K. Koo. Interpreting cis-regulatory interactions from large-scale deep neural networks. *Nature Genetics*, 56(11):2517–2527, November 2024. ISSN 1546-1718. doi: 10.1038/s41588-024-01923-3.

Mandy S. Wong, Justin B. Kinney, and Adrian R. Krainer. Quantitative Activity Profile and Context Dependence of All Human 5′ Splice Sites. *Molecular Cell*, 71(6):1012–1026.e3, September 2018. ISSN 1097-2765. doi: 10.1016/j.molcel.2018.07.033.

Gene Yeo and Christopher B. Burge. Maximum Entropy Modeling of Short Sequence Motifs with Applications to RNA Splicing Signals. *Journal of Computational Biology*, 11(2-3):377–394, March 2004. ISSN 1066-5277. doi: 10.1089/1066527041410418.

Juannan Zhou, Mandy S. Wong, Wei-Chia Chen, Adrian R. Krainer, Justin B. Kinney, and David M. McCandlish. Higher-order epistasis and phenotypic prediction. *Proceedings of the National Academy of Sciences*, 119(39):e2204233119, September 2022. doi: 10.1073/pnas.2204233119.

Valentin Zulkower and Susan Rosser. DNA chisel, a versatile sequence optimizer. *Bioinformatics*, 36(16):4508–4509, 2020. ISSN 1367-4803. doi: 10.1093/bioinformatics/btaa558.